Fatec
Ipiranga
Pastor Eneas Tognini
.
TADS - Análise Desenvolvimento de Sistemas

Programa Estruturara e Modular

Aula #06
Recursividade

Prof.: Veríssimo - carlos.pereira70@fatec.sp.gov.br

21/03/2025

Sobre este documento

Este documento objetiva orientar o aluno quanto ao conteúdo a ser desenvolvido nesta disciplina, bem como a divulgação dos critérios de avaliação que serão empregados.

É muito importante que o aluno faça uma acurada leitura deste documento, pois nele conterá a descrição dos elementos que norteiam o curso.

Contents

1 Recursividade **1**

1.1 Introdução 1

1.2 Conceitos 1

1.2.1 Particularidades 2

1.3 Definições Recursivas 4

1.4 Definições Recursivas - Exemplo 5

1.4.1 Sequência de Fibonacci 6

1.5 Memória na recursividade 8

1.5.1 Pilha de Chamadas (Call Stack) 8

1.5.2 Comparação com a Iteração 9

Chapter 1

Recursividade

1.1 Introdução

Recursão é o processo de definição de algo em termos de si mesmo (Laureano,2008). Um algoritmo recursivo é aquele que utiliza a si próprio para atingir algum objetivo, ou obter algum resultado. Em programação, a recursividade está presente quando a função realiza chamadas a si mesma.

1.2 Conceitos

Ao desenvolver um algoritmo,muitas vezes precisamos executar uma tarefa repetidamente, utilizando para isso estruturas de repetição ***para*** ou ***enquanto***, ou precisamos tomar decisões condicionais, utilizando operações da forma "*se* ::: *senão* ::: *então*". Em geral, todas essas operações são rapidamente assimiladas pois fazem parte do cotidiano de qualquer pessoa, dado que muitas vezes precisamos tomar decisões condicionais ou executar tarefas repetidamente. Porém, para desenvolver alguns algoritmos é necessário fazer uso da **re-**

cursão.

Qualquer **função que chama a si mesma** é recursiva, mas **recursão é muito mais do que isso**. Ela é uma técnica muito poderosa para solução de problemas. A ideia é reduzir uma instância em instâncias menores, do mesmo problema, que por sua vez também são reduzidas, e assim por diante, até que elas sejam tão pequenas que possam ser resolvidas diretamente.

Diversos problemas têm essa característica: toda instância contém uma instância menor do mesmo problema (estrutura recursiva). Mas veja que não basta apenas reduzir o tamanho da instância. Deve-se encontrar uma instância menor (ou até mais de uma) cuja solução ajude a instância maior a ser resolvida.

Um algoritmo recursivo é muito empregado no conceito chamado "dividir para conquistar" para resolver um problema. Ele divide um problema maior em problemas menores. Estas pequenas partes são solucionadas de forma mais rápida e mais eficiente. Todas as pequenas soluções são, portanto, agregadas para obter a solução maior e final para o problema.

1.2.1 Particularidades

Em uma definição recursiva um item é definido em termos de si mesmo, ou seja, o item que está sendo definido aparece como parte da definição.

Em todas as funções recursivas existe:

- Caso base (um ou mais) cujo resultado é imediatamente conhecido.
- Passo recursivo em que se tenta resolver um sub-problema do problema inicial.

- As funções recursivas são em sua maioria soluções mais elegantes e simples, se comparadas a funções tradicionais ou iterativas, já que executam tarefas repetitivas sem utilizar nenhuma estrutura de repetição, como *for* ou *while*. Porém essa elegância e simplicidade têm um preço que requer muita atenção em sua implementação.

- Trazendo a recursividade para o nosso cotidiano, um ótimo exemplo está na ação de contar um saco de moedas, onde a cada ato de retirar uma moeda do saco precisa-se "contar dinheiro" que corresponde a identificar qual é o valor da moeda e somá-la à quantia que ainda está no saco.

  - Para identificar a quantia que ainda está no saco basta chamar a mesma função "contar dinheiro" novamente, porém dessa vez já considerando que essa moeda não está mais lá. E este processo de retirar uma moeda, identificar seu valor e somar com o restante do saco se repete até que o saco esteja vazio, quando atingiremos o ponto de parada e a função retornará o valor zero, indicando que não há mais moedas no saco.

1.3 Definições Recursivas

Recursividade

Uma função recursiva é aquela que faz uma chamada a si mesma.Essa chamada pode ser:

- Direta: Uma função A chama a ela pópria
- Indireta: A função A chama f função B, que, por sua vez, chama Uma função A

O Algoritmo 1.2 mostra a implementação de uma recursividade direta

```
void func_rec(int n)
{
    ...
    func_rec(n - 1);
    ...
}
```

Listing 1.1: Recursividade Direta

1.4 Definições Recursivas - Exemplo

Abordamos o exemplo de função recursiva para cálculo do fatorial de um determinado número.:

$$n! = \begin{cases} 1, \; se \; n = 0 \\ n \times (n-1)!, \; se \; n > 0 \end{cases}$$

```
/* Função recursiva para cálculo do fatorial */
int fat (int n)
{
   if (n==0)
      return 1;
   else
      return n*fat(n-1);
}
```

Caso BASE

Passo Recursivo

$$fatorial(n) = \begin{cases} 1 & \text{se } n = 0 \\ n \times fatorial(n-1) & \text{se } n > 0 \end{cases}$$

Para calcularmos o fatorial de 3! temos o seguinte resultado:

$$\begin{aligned} & fatorial(3) = 3 * fatorial(3-1) \\ & = 3 * fatorial(2) \\ & = 3 * 2 * fatorial(2-1) \\ & = 3 * 2 * fatorial(1) \\ & = 3 * 2 * 1 * fatorial(1-1) \\ & = 3 * 2 * 1 * fatorial(0) \\ & = 3 * 2 * 1 * 1 \\ & = 6 \end{aligned}$$

1.4.1 Sequência de Fibonacci

É uma sequência de números inteiros, começando normalmente por 0 e 1, na qual cada termo subsequente corresponde à soma dos dois anteriores. A sequência recebeu o nome do matemático italiano *Leonardo de Pisa* (*Leonardo Fibonacci*). Esta seqência descreveu (1202) o crescimento de uma população de coelhos.

$$\boxed{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...}$$

A listagem 1.2 mostra um exemplo de solução da Sequência de Fibonacci, utilizando o paradigma de laço de repetição ***for***:

```
1  #include <stdio.h>
2
3  void fibonacci_iterativo(int n) {
4     int a = 0, b = 1, temp;
5      printf("Os primeiros %d termos da sequ ncia
          de Fibonacci sao: ", n);
6      for (int i = 0; i < n; i++) {
7          printf("%d ", a);
8          temp = a;
9          a = b;
10         b = temp + b;
11     }
12     printf("\n");
13 }
14
15 int main() {
16     int termos = 10;
17     fibonacci_iterativo(termos);
18     return 0;
19 }
```

Listing 1.2: Sequência de Fibonacci - utilizando o paradigma de laço de repetição

A listagem 1.3 mostra um exemplo de solução da Sequência de Fibonacci, utilizando o paradigma ***Recursividade***:

```
#include <stdio.h>

void fibonacci_recursivo(int termosProcessados,
    int a, int b) {
    if (termosProcessados > 0) {
        printf("%d ", a);
        fibonacci_recursivo(termosProcessados - 1,
            b, a + b);
    }
}

int main() {
    int termos = 10;
    printf("Os primeiros %d termos da sequ ncia
        de Fibonacci  s  o: ", termos);
    fibonacci_recursivo(termos, 0, 1);
    printf("\n");
    return 0;
}
```

Listing 1.3: Sequência de Fibonacci - utilizando o paradigma Recursividade

1.5 Memória na recursividade

A recursividade é uma técnica poderosa em programação, mas seu uso tem implicações significativas no consumo de memória. Quando uma função recursiva é chamada, várias operações ocorrem em termos de alocação de memória, que são fundamentais para entender como a recursividade funciona internamente.

1.5.1 Pilha de Chamadas (Call Stack)

- Contexto de Execução:
  - Cada vez que uma função recursiva é chamada, um novo contexto de execução é criado. Esse contexto inclui:
    * Parâmetros da Função: Valores passados para a função.
    * Variáveis Locais: Variáveis definidas dentro da função.
    * Endereço de Retorno: Local na memória onde a execução deve continuar após a função terminar.
- Alocação na Pilha:
  - Esses contextos de execução são armazenados na pilha de chamadas (call stack). A pilha é uma estrutura de dados que segue o princípio LIFO (Last In, First Out), onde o último contexto de execução adicionado é o primeiro a ser removido.
- Consumo de Memória:
  - Cada chamada recursiva adiciona um novo quadro (frame) à pilha. Se a profundidade da recursão

for muito grande, a pilha pode crescer significativamente, consumindo uma quantidade considerável de memória. Isso pode levar a um estouro de pilha (stack overflow) se a memória disponível for esgotada.

1.5.2 Comparação com a Iteração

- Iteração:
  - Em uma abordagem iterativa, o uso de memória é geralmente mais eficiente. As variáveis são reutilizadas em cada iteração do laço, e não há necessidade de alocar novos contextos de execução repetidamente.
- Recursividade:
  - Embora a recursividade possa tornar o código mais elegante e fácil de entender, ela pode ser menos eficiente em termos de memória. Cada chamada recursiva adiciona sobrecarga à pilha de chamadas, o que pode ser problemático para recursões profundas.